

HaulCheck — handover

Everything you need to take ownership of the application: what it was, what changed, and the steps only you can do because they need accounts in your name.

If you only read one section, read [Part 3: Your setup checklist](#).

Part 1 — What the app was

HaulCheck is a road-haulage compliance platform for UK (DVSA) and Ireland (RSA) transport operators. It tracks vehicle MOT/CVRT, tax, service and PMI inspections; defects and walkaround checks; driver licences, CPC and tachograph hours; operator licence and insurance. It scores compliance, raises alerts before things expire, and produces PDF audit packs. Fleet managers use a desktop dashboard; drivers use a mobile app for walkarounds and defect reports.

All of that already worked. The problem was not the features — it was that the application could not be **sold to more than one customer**, and could not be **run by anyone other than the platform it was built on**.

Four things stood in the way:

- 1. "Customer" meant "one person."** There was no concept of a company. When a manager invited a colleague, the colleague got their own separate, empty account. Two managers at the same haulage firm could not see the same fleet. There was no way to have a customer with five staff.
- 2. There was no way for you to administer it.** No admin role, no way to see who your customers were, no way to suspend an account that stopped paying, no record of who did what. The only platform-wide figure that existed was a count of all registered users — and it was being displayed on *every customer's* dashboard.
- 3. Sign-in was not hardened for public signup.** Passwords needed only six characters. There was no limit on login attempts, so an attacker could guess indefinitely. The driver login accepted a 6-character code checked across *every* customer at once, with no rate limit — so guessing codes at random would eventually log you in as somebody's driver. Email addresses were never verified.
- 4. Nothing was ready for load.** The database had **no indexes at all**, so every screen read every record of every customer. Scheduled reminder emails ran inside the web process, meaning that running two copies of the app would send every customer two copies of every reminder.

And underneath all of it: **the app was wired to the Emergent platform**. File uploads, all AI features and Google sign-in ran through Emergent's own services using an Emergent key. On your own hosting, none of those would work.

Part 2 — What changed

Seven stages, each committed separately so the history is reviewable.

Organisations — customers are now companies

A company is now the unit of ownership. Staff belong to it with a role:

Role	Can do
Owner	Everything, including company settings and inviting staff
Manager	All fleet and compliance data; not company settings
Viewer	Read-only — cannot change anything

Every record in the system is tagged with its company, and every database query filters on it. Inviting a colleague now has two clearly separate options: **join my company** (shared fleet) or **set up their own account** (the old behaviour, kept because it was in use).

Existing data was migrated by a script that is safe to re-run and was tested in dry-run mode first. Nobody's data was merged: every existing account became its own company, exactly as isolated as before.

How this is kept safe: a test reads the source code and fails the build if anyone writes a query that filters by **person** instead of by **company**. This matters because a missing filter does not cause a visible error — the page still loads, just with another customer's records on it. That test has already caught three real mistakes.

Sign-in security

- Passwords must be 12+ characters and are checked against common weak patterns.
- Login attempts are rate-limited with logout, per account **and** per address.
- Driver access codes are rate-limited too, closing the guessing hole.
- Email addresses are verified.
- Suspending someone now cuts off their existing session immediately. Previously

they stayed logged in until their token expired — up to 30 days.

- The app refuses to start in production with a wide-open cross-origin setting.

One deliberate design note: the per-address limits are much looser than the per-account ones. An entire transport office shares one internet address, so limits tight enough for a single account would lock out the whole company.

Your admin console

At `/admin`, visible only to you:

- **Tenants** — every customer, their staff, how much data they hold; suspend, reactivate or delete.

- **Users** — search across all customers, suspend, force a password reset.
- **Metrics** — signups over time, active companies, storage per customer.
- **Impersonation** — sign in as a customer to support them. **Read-only:** you

can see their screens but cannot change their data. A banner shows it, and every action is logged.

- **Audit log** — append-only. Cannot be edited or deleted through any API.

Admin access cannot be granted through the website by anyone, including you. It is set by running a command against the database (see Part 3). This is deliberate: it means no bug in the signup flow can ever hand someone admin.

The all-customers user count was removed from the customer dashboard and now lives here, where it belongs.

Off the Emergent platform

File storage, AI and email now sit behind interfaces, so the vendor is a configuration choice rather than something baked into the code:

Service	Default	You can switch to
File storage	Emergent	Amazon S3, Cloudflare R2, MinIO, Backblaze
AI	Emergent	Anthropic (Claude) directly
Email	Resend	any provider, by adding one class

If you configure nothing, the app still starts and runs. Each unconfigured service reports itself clearly instead of crashing, and every non-AI, non-upload feature works normally. This means you can deploy first and add keys as you get them.

Google sign-in previously ran through Emergent's shared demo server, meaning your users' identities depended on a third party you have no account with and cannot control. It now talks to Google directly using your own credentials.

Speed and scale

- **59 database indexes** added. Listing one company's vehicles used to read every vehicle belonging to every customer; now it reads only that company's.
- Email addresses are now enforced unique **by the database**. Previously two simultaneous signups could create two accounts for the same address.
- Scheduled reminders now use a lock, so running several copies of the app sends each reminder once.
- The dashboard made ~20 database round trips one after another; they now run together.
- `/api/health` reports readiness and returns an error status when the database

is unreachable, so hosting platforms can detect a broken instance.

- Long lists now page properly instead of silently cutting off at 1,000 records.

Testing

	Before	After
Passing	221	372

Every stage was checked against the previous one, and no stage was allowed to break anything that previously worked. The remaining failures are listed and explained in `docs/TEST_BASELINE.md` — they need API keys, or they are old tests that disagree with deliberate product changes.

Part 3 — Your setup checklist

These need accounts in your name. Nothing here can be done for you.

Required

1. A server and a database

You need somewhere to run the app and a MongoDB database. MongoDB Atlas has a free tier that is enough to start. Anything that runs Python and Node works for hosting — Railway, Render, DigitalOcean, AWS.

You will need the connection string, which looks like:

```
mongodb+srv://user:password@cluster.mongodb.net
```

2. A signing secret

Generate one and keep it private. It signs login sessions — anyone who has it can forge a login.

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Changing it later logs everyone out. That is the correct way to respond if you ever think it has leaked.

3. Fill in the configuration

Copy `backend/.env.example` to `backend/.env` and fill it in. Every setting is commented in that file. The ones you must set:

Setting	What it is
MONGO_URL	Database connection string from step 1
DB_NAME	Any name, e.g. <code>haulcheck</code>
JWT_SECRET	The secret from step 2
CORS_ORIGINS	Your website address, e.g. <code>https://app.yourdomain.com</code>
ENVIRONMENT	<code>production</code> when live

Then copy `frontend/.env.example` to `frontend/.env` and set `REACT_APP_BACKEND_URL` to your API address.

⚠ **Never commit a real `.env` file.** They are excluded from version control already. If a key is ever committed or pasted somewhere public, treat it as compromised and regenerate it — that applies to all of the keys below.

4. Make yourself the administrator

Register normally through the website first, then run:

```
cd backend
python scripts/grant_admin.py your@email.com
```

`/admin` appears after you sign in again. As above, there is no way to do this through the website — that is the point.

Optional — each unlocks one feature

The app runs without all of these. Add them when you want the feature.

File uploads and attachments

Needed to attach photos to defects, upload documents, and include attachments in audit packs.

Use **Cloudflare R2** or **Amazon S3**. R2 is usually cheaper and has no charge for downloads.

1. Create a bucket.
2. Create an access key with read/write on that bucket.
3. Set `STORAGE_PROVIDER=s3` and fill in `S3_BUCKET` , `S3_ENDPOINT` ,

`S3_ACCESS_KEY` , `S3_SECRET_KEY` .

AI features

Needed for defect summaries, letter drafting, insurance document reading, tachograph analysis and the fleet risk briefing.

1. Create an account at [<https://console.anthropic.com>](https://console.anthropic.com).

2. Add billing and create an API key.
3. Set `AI_PROVIDER=anthropic` and `ANTHROPIC_API_KEY`.

This is billed per use. Set a spending limit in the Anthropic console.

Email

Needed for defect alerts, expiry reminders, invitations, password resets and sending audit packs.

1. Create an account at [<https://resend.com>](https://resend.com).
2. **Verify your sending domain** — this is the step people skip, and without it

your email lands in spam or is rejected. Resend gives you DNS records (SPF, DKIM) to add wherever your domain is registered.

1. Create an API key.
2. Set `RESEND_API_KEY` and `SENDER_EMAIL` (an address at your verified domain).

Google sign-in

Optional. Email and password sign-in works without it, and the Google button is hidden entirely when it is not configured.

1. Go to [<https://console.cloud.google.com>](https://console.cloud.google.com), create a project.
2. **APIs & Services** → **Credentials** → **Create OAuth client ID** → **Web application**.
3. Under **Authorised redirect URIs**, add exactly:

`https://yourdomain.com/auth/google/callback` — the path must match precisely, and the domain must also be in `CORS_ORIGINS`. A mismatch here is the single most common setup failure; the server logs Google's exact complaint when it happens.

1. Copy the client ID and secret into `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`.
2. To publish beyond your own account, complete Google's consent screen. While

it is in "testing", only accounts you list can sign in.

If you are behind a load balancer or CDN

Set `TRUST_PROXY_HEADERS=1` **only** if your app sits behind a proxy (Cloudflare, nginx, a cloud load balancer) that overwrites the client address header.

If you set it without such a proxy, anyone can forge their address and bypass the login rate limits entirely. If you **don't** set it when you do have one, every visitor looks like the same address and they will lock each other out.

Part 4 — Running it

Full detail in `docs/DEVELOPMENT.md`. Briefly:

```
# Backend
cd backend
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements-local.txt # bin/ on Mac/Linux
.venv/Scripts/python -m uvicorn server:app --port 8000

# Frontend
cd frontend
yarn install
yarn build # yarn start for development
```

Use `requirements-local.txt`, not `requirements.txt` — the latter is Emergent's deployment manifest and pins their whole platform image.

Check it is alive at `/api/health`. Indexes are created automatically on first start; you will see `Indexes ready: 59 created/verified` in the log.

Before going live

- ☐ `ENVIRONMENT=production`
- ☐ `CORS_ORIGINS` lists your real domain, not `*`
- ☐ `JWT_SECRET` is a fresh generated value, not the example
- ☐ HTTPS enabled (sessions use secure cookies)
- ☐ Database backups turned on — Atlas does this for you
- ☐ You have run `grant_admin.py` and can reach `/admin`

Part 5 — Known limitations

Stated plainly so nothing is a surprise later.

Billing is not built. The company record already reserves the fields for a plan, plan limits and subscription status, so payments can be added without a second data migration. No payment provider is connected.

Some old tests fail, on purpose. `docs/TEST_BASELINE.md` explains each one. Four of them fail because the `*test*` is out of date, not the app — do not "fix" the app to satisfy them.

One real bug is documented but unfixed: dates are calculated in UTC while some tests use local time, so around midnight in a non-UTC timezone a compliance due date can be off by one day. It is described in `docs/TEST_BASELINE.md`.

backend/server.py is large (~5,000 lines). It works and is well covered by tests; splitting it into modules is a tidy-up, not a fix.

The `emergentintegrations` package is not on PyPI. If you keep Emergent as your AI provider you can only install it inside their image. Switching `AI_PROVIDER=anthropic` removes that constraint — which is the recommended path.